Faculty of Engineering & Technology Department of Electrical & Computer Engineering ENCS3390: Operating System Concepts First Semester,2024/202

Project 2

Prepared by:

Name: Shahd Raed Shwekeyeh & Nadeen Ali Jaber

Number : 1222105 & 1220044

Instructor: Mohammad Khalil

Abstract

This project simulates a CPU scheduling system with deadlock detection and recovery, focusing on a single CPU core and multiple resource types. The system implements priority scheduling with round-robin and handles resource requests and releases. Deadlock detection is incorporated, and recovery strategies such as process termination or resource preemption are applied when deadlocks are detected. The simulation tracks the execution timeline in a Gantt chart and calculates average waiting and turnaround times, providing insights into CPU scheduling, I/O management, and deadlock resolution in operating systems.

1.Reading from file

File content:

0 0 1 CPU {R[1], 50, F[1]}
1 5 1 CPU {20} IO{30} CPU{20, R[2], 30, F[2], 10}

output:

```python
def display_processes(processes):
    for process in processes:
        print(f"Process ID: {process.pid}")
        print(f"Arrival Time: {process.arrival_time}")
        print(f"Priority: {process.priority}")
        print(f"CPU Bursts: {process.cpu_bursts}")
        print(f"IO Bursts: {process.io_bursts}")
        print(f"Resource Request: {process.resource_request}")
        print(f"Resource Release: {process.resource_release}")
        print()
processes = read_input_file("input.txt")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

```
Arrival Time: 0
Priority: 1
CPU Bursts: [50]
IO Bursts: []
Resource Request: [1]
Resource Release: [1]

Process ID: 1
Arrival Time: 5
Priority: 1
CPU Bursts: [20, 20, 30, 10]
IO Bursts: [30]
Resource Request: [2]
Resource Release: [2]

PS C:\Users\shahed\Desktop\python>
```

2. Non preemptive Priority Scheduling

File content:

0 0 1 CPU{50} IO{30} CPU{20}
1 5 1 CPU{20} IO{30} CPU{20}
2 5 0 CPU{20}

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                   Python

PS C:\Users\shahed\Desktop\python> & C:/Users/shahed/AppData/Local/Programs/Python/Python313/python.exe c:/Users/shahed/Desktop/python/sh
Gantt Chart:
P0[0-10] P2[10-20] P2[20-30] P1[30-40] P0[40-50] P1[50-60] P0[60-70] P0[70-80] P0[80-90] P1[90-100] P1[100-110] P0[120-130] P0[130-140]
Average Waiting Time: 26.666666666666668
Average Turnaround Time: 90.0
PS C:\Users\shahed\Desktop\python>
```

Gantt chart

P0	P2	P2	P1	P0	P1	P0	P0	P0	P1	P1	–	P0	P0

0 10 20 30 40 50 60 70 80 90 100 110 120 130 140

waiting Queue

P1	P1	P1	P0

60 70 80 90 120

P1: (waiting time = (ready Queue time).
= turnaround − cpubursts − I/O burst
= 5 + 20 + 10 = 35

(P1) turnaround = 110 − 5 = 105

P2: waiting = 5
P2: around = 30 − 5 = 25

P0: −
turn around = 140
waiting = 30 + 10 + 10 = 50

Avg Turnaround = (105 + 25 + 140) / 3 = 90
Avg waiting = (5 + 35 + 50)/3 = 26.6666

Code:

```
import re
from collections import deque

class Process:
    def __init__(self, pid, arrival_time, priority):
        self.pid = pid
        self.arrival_time = arrival_time
        self.priority = priority
        self.cpu_bursts = []  # List to store CPU bursts
        self.io_bursts = []  # List to store IO bursts
        self.remaining_cpu = 0  # Remaining CPU burst time
        self.wait_time = 0  # Waiting time
        self.turnaround_time = 0  # Turnaround time
        self.completed_bursts = 0  # Number of bursts completed
        self.completion_time = 0  # Completion time

    def add_burst(self, cpu_burst, io_burst):
        self.cpu_bursts.append(cpu_burst)
        self.io_bursts.append(io_burst)
        if self.remaining_cpu == 0:  # Initialize remaining CPU time on
first burst
            self.remaining_cpu = cpu_burst

def parse_input(file_path):
    processes = []
    with open(file_path, "r") as file:
        for line in file:
            if not line.strip():
                continue
            parts = re.split(r'\s+', line.strip(), maxsplit=3)
            if len(parts) < 4:
                print(f"Skipping invalid line: {line.strip()}")
                continue
            try:
                pid = int(parts[0])
                arrival_time = int(parts[1])
                priority = int(parts[2])
                bursts = parse_bursts(parts[3])
                process = Process(pid, arrival_time, priority)
```

```
                for i in range(0, len(bursts), 2):
                    cpu_burst = bursts[i][1]
                    io_burst = bursts[i + 1][1] if i + 1 < len(bursts)
else 0
                    process.add_burst(cpu_burst, io_burst)
                processes.append(process)
            except ValueError as e:
                print(f"Error parsing line: {line.strip()} -> {e}")
                continue
    return processes

def parse_bursts(burst_string):
    bursts = []
    burst_entries = re.findall(r"(CPU|IO)\{(\d+)\}", burst_string)
    for entry in burst_entries:
        burst_type, time = entry
        bursts.append((burst_type, int(time)))
    return bursts

def print_gantt_chart(gantt_chart):
    gantt_string = ""  # Initialize a string to accumulate the Gantt chart
output
    for pid, start, end in gantt_chart:
        gantt_string += f"P{pid}[{start}-{end}] "  # Append each process
with its execution time
    print(gantt_string.strip())  # Print the entire Gantt chart on one
line, removing extra space at the end

def non_preemptive_priority_scheduling(processes, time_quantum):
    processes.sort(key=lambda x: x.arrival_time)  # Sort by arrival time

    time = 0  # Current time in the scheduling process
    gantt_chart = []  # List to hold the Gantt chart
    completed_processes = []  # List of completed processes
    ready_queue = deque()  # Ready queue for processes
    waiting_queue = []  # List of processes waiting for IO to complete
    process_index = 0  # Index to iterate through processes

    while len(completed_processes) < len(processes):
```

```
        # Add processes that have arrived to the ready queue
        while process_index < len(processes) and
processes[process_index].arrival_time <= time:
            ready_queue.append(processes[process_index])
            process_index += 1

        # Move processes from waiting queue to ready queue if IO burst has
completed
        for io_end_time, process in list(waiting_queue):
            if io_end_time <= time:
                waiting_queue.remove((io_end_time, process))
                process.remaining_cpu =
process.cpu_bursts[process.completed_bursts]
                ready_queue.append(process)

        if ready_queue:
            ready_queue = deque(sorted(ready_queue, key=lambda x:
x.priority))  # Sort by priority
            current_priority = ready_queue[0].priority
            same_priority_queue = deque([p for p in ready_queue if
p.priority == current_priority])
            ready_queue = deque([p for p in ready_queue if p.priority !=
current_priority])

            while same_priority_queue:
                process = same_priority_queue.popleft()
                start_time = time
                cpu_time = min(process.remaining_cpu, time_quantum)

                time += cpu_time
                gantt_chart.append((process.pid, start_time, time))
                process.remaining_cpu -= cpu_time

                # Add processes to the ready queue as time progresses
                while process_index < len(processes) and
processes[process_index].arrival_time <= time:
                    ready_queue.append(processes[process_index])
                    process_index += 1
```

```
                ready_queue = deque(sorted(ready_queue, key=lambda x:
x.priority))  # Re-sort after adding new processes

                if ready_queue and ready_queue[0].priority <
current_priority:
                    same_priority_queue.appendleft(process)
                    break

                if process.remaining_cpu == 0:
                    process.completed_bursts += 1
                    if process.completed_bursts < len(process.cpu_bursts):
                        io_burst =
process.io_bursts[process.completed_bursts - 1]
                        io_end_time = time + io_burst
                        waiting_queue.append((io_end_time, process))
                    else:
                        process.completion_time = time
                        process.turnaround_time = time -
process.arrival_time
                        process.wait_time = process.turnaround_time -
(sum(process.cpu_bursts) + sum(process.io_bursts))
                        completed_processes.append(process)
                else:
                    same_priority_queue.append(process)

            ready_queue.extend(same_priority_queue)
        else:
            time += 1  # Increment time if the ready queue is empty

    avg_waiting_time = sum(p.wait_time for p in completed_processes) /
len(completed_processes)
    avg_turnaround_time = sum(p.turnaround_time for p in
completed_processes) / len(completed_processes)

    return gantt_chart, avg_waiting_time, avg_turnaround_time

# Example usage
file_path = "input.txt"
time_quantum = 10  # Time quantum for round-robin aspect
processes = parse_input(file_path)
```

```python
gantt_chart, avg_waiting_time, avg_turnaround_time =
non_preemptive_priority_scheduling(processes, time_quantum)

print("Gantt Chart:")
print_gantt_chart(gantt_chart)
print("Average Waiting Time:", avg_waiting_time)
print("Average Turnaround Time:", avg_turnaround_time)
```

Second test case:
0 0 1 CPU{5} IO{3} CPU{6}
1 0 2 CPU{4} IO{2} CPU{5}
2 0 3 CPU{3} IO{1} CPU{4}

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                    Python

PS C:\Users\shahed\Desktop\python> & C:/Users/shahed/AppData/Local/Programs/Python/Python313/python.exe c:/Users/shahed/Desktop/python/shah2
Gantt Chart:
P0[0-5] P1[5-9] P0[9-15] P1[15-20] P2[20-23] P2[24-28]
Average Waiting Time: 10.0
Average Turnaround Time: 21.0
PS C:\Users\shahed\Desktop\python>
```

THE END